

Introduction to Algorithmic Differentiation

Everything you ever wanted to know but were afraid to ask

Jean-Luc Bouchot (INRIA Saclay)

Sri Hari Krishna Narayanan (Argonne National Laboratory)

June 23, 2026

Outline

- 1 Motivation
- 2 Real valued functions
- 3 Forward Mode
- 4 Multivalued inputs or outputs
- 5 Reverse Mode
- 6 Pitfalls
- 7 Summary
- 8 Tapenade Practice

What can you expect from this tutorial?

Participants will:

- Understand the basics of algorithmic differentiation (AD) with respect to other approaches (finite differences, symbolic differentiation)
- Learn how to use AD tools for differentiating
- See and implement examples of forward and reverse mode AD
- Discuss common pitfalls and best practices for self diagnosis
- ... maybe even want to get involved in R & D of AD tools!

What we expect from you?

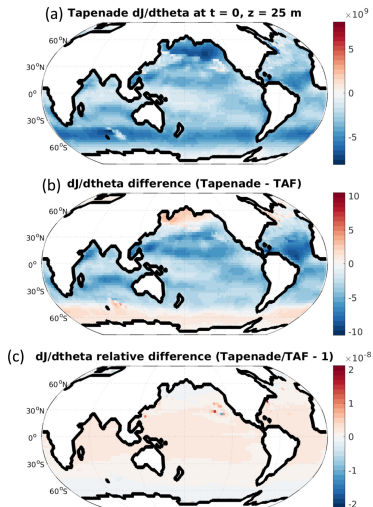
This is a diverse crowd at JDEV. We assume some basic understanding of the following, although we try to review the most important background material:

- Basic calculus (chain rule, partial derivatives)
- Basic programming (variables, loops, functions)
- Some familiarity with Fortran/C (but we will try to keep it as language-agnostic as possible)
- Some familiarity of numerical methods
- Some familiarity with optimization and machine learning (but not required)

What are some successful uses of AD?

MITgcm

Compute XXX



What are some successful uses of AD?

HBTHO

Compute XXX

FIGS/a-picture-for-hfbtho.

Why Do We Need Derivatives?

Common tasks:

- Optimization loop
- Sensitivity analysis
- Data assimilation
- Machine learning
- PDE-constrained problems

Key question:

How do we compute derivatives of *programs* accurately and **efficiently**?

Three Approaches (1/3)

Finite Differences

- Easy
- Numerical error
- Expensive in high dimension
- Parametrization

Example

Let $f : \mathbb{R}^p \rightarrow \mathbb{R}$. Each derivative implies the computation of

$$\frac{\partial f}{\partial \mathbf{x}_i} \approx \frac{f(\mathbf{x} + \varepsilon \mathbf{e}_i) - f(\mathbf{x} - \varepsilon \mathbf{e}_i)}{[2]\varepsilon}$$

Computing the gradient costs $[2\times]p$ the cost of a single function evaluation.

Three Approaches (2/3)

Symbolic Differentiation

- Exact
- Expression swell
- Hard for real codes

Example

Consider the function $f : \mathbb{R} \rightarrow \mathbb{R}$ defined as $f(x) = x^2 \sin(x^2/2 + 3x)$. Its symbolic derivative is given by

$$\frac{df}{dx}(x) = 2x \sin(x^2/2 + 3x) + x^2(x + 3) \cos(x^2/2 + 3x)$$

Three Approaches (3/3)

Algorithmic Differentiation

- Exact up to machine precision
 - Works on programs
 - Efficient
-
- AD differentiates code, not formula!
 - AD factorizes / reuses variables

Forward diff by dual numbers

Dual numbers

Let every number embed its differentiated value. Define:

$$\mathbf{x} = x + \dot{x}\varepsilon$$

with the fictitious number $\varepsilon^2 = 0$.

ε is not *some small number*, but redefines the arithmetic on real numbers.

Dual number propagation

We propagate derivatives together with values.

$$\mathbf{x} \rightarrow (x, \dot{x})$$

Each operation updates both.

Dual number propagation

We propagate derivatives together with values.

$$\mathbf{x} \rightarrow (x, \dot{x})$$

Each operation updates both.

Example

Consider the univariate function $f(x) = x^2$.

$$\mathbf{x}^2 = (x + \dot{x}\epsilon)^2 = x^2 + 2x\dot{x}\epsilon \left[+\dot{x}^2\epsilon^2 \right].$$

Key insight

Forward AD is ordinary arithmetic on dual numbers.

Forward Mode Example

Given the function $z = f(x) = \sin(x^2)$

Propagation table:

Variable	Value	Derivative
x	x	$\dot{x}$
$y = x^2$	x^2	$2x\dot{x}$
$z = \sin(y)$	$\sin(y)$	$\cos(y) \cdot 2x\dot{x}$

Let's try

- $f(x) = \sin(x)$
- $f(x) = \cos(x)$
- $f(x) = \sqrt{x}$

Let's try

sin

$$\begin{aligned}\sin(\mathbf{x}) &= \sin(x) \cos(\dot{x}\varepsilon) + \cos(x) \sin(\dot{x}\varepsilon) \\ &= \sin(x) (1 + \mathcal{O}(\dot{x}^2\varepsilon^2)) + \cos(x)(\dot{x}\varepsilon + \mathcal{O}(\dot{x}^3\varepsilon^3)) \\ &= \sin(x) + \cos(x)\dot{x}\varepsilon\end{aligned}$$

COS

$$\begin{aligned}\cos(\mathbf{x}) &= \cos(x) \cos(\dot{x}\varepsilon) - \sin(x) \sin(\dot{x}\varepsilon) \\ &= \cos(x) (1 + \mathcal{O}(\dot{x}^2\varepsilon^2)) - \sin(x)(\dot{x}\varepsilon + \mathcal{O}(\dot{x}^3\varepsilon^3)) \\ &= \cos(x) - \sin(x)\dot{x}\varepsilon\end{aligned}$$

$\sqrt{\cdot}$

$$\sqrt{\mathbf{x}} = \sqrt{x} \sqrt{1 + \frac{\dot{x}}{x}\varepsilon} = \sqrt{x} \left(1 + \frac{1}{2} \frac{\dot{x}}{x} \varepsilon + \mathcal{O}(\dot{x}^2\varepsilon^2) \right)$$

Differential examples

Differentiating elementary operations:

Operations	Primal	Derivative
add	$z = x + y$	$dz = dx + dy$
mul	$z = x \times y$	$dz = ydx + xdy$
sin	$z = \sin(x)$	$dz = \cos(x)dx$

Programs as Computational Graphs

INPUT: x

OUTPUT: y, z, r

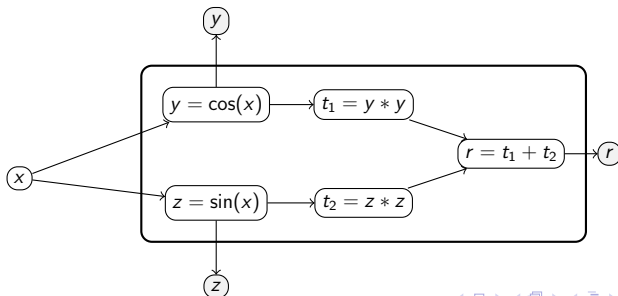
$y = \cos(x)$

$z = \sin(x)$

$t1 = y * y$

$t2 = z * z$

$r = t1 + t2$



Forward AD via Dual Numbers

Seed the input perturbation with $\mathbf{x} = x + 1 \cdot \varepsilon$, $\varepsilon^2 = 0$.

Propagate through the program:

$$\begin{aligned}\mathbf{y} &= \cos(\mathbf{x}) = \cos(x) - \sin(x)\varepsilon, & \dot{y} &= -\sin(x), \\ \mathbf{z} &= \sin(\mathbf{x}) = \sin(x) + \cos(x)\varepsilon, & \dot{z} &= \cos(x), \\ \mathbf{t}_1 &= \mathbf{y}^2 = y^2 + 2y\dot{y}\varepsilon, & \dot{t}_1 &= 2y\dot{y}, \\ \mathbf{t}_2 &= \mathbf{z}^2 = z^2 + 2z\dot{z}\varepsilon, & \dot{t}_2 &= 2z\dot{z}, \\ \mathbf{r} &= \mathbf{t}_1 + \mathbf{t}_2 = r + (\dot{t}_1 + \dot{t}_2)\varepsilon & \dot{z} &= \cos(x), \dot{r} = \dot{t}_1 + \dot{t}_2.\end{aligned}$$

This is AD

```
SUBROUTINE FOO(x,    y,    z,    r,    )
```

```
REAL x,y,z,r
```

```
y = COS(x)
```

```
z = SIN(x)
```

```
r = y*y + z*z
```

```
END
```

This is AD

```

SUBROUTINE FOO(x, xd, y, yd, z, zd, r, rd)
  REAL yd, zd, rd, xd
  REAL x, y, z, r
  yd = -SIN(x)*xd
  y = COS(x)
  zd = COS(x)*xd
  z = SIN(x)
  rd = 2*y*yd + 2*z*zd
  r = y*y + z*z
END
```

Forward Mode review

Forward / tangent mode idea

Forward AD propagates perturbations: "How much is this output influenced by a small change in this input?"

Source transformation AD

Systematic application of the chain rule on every instruction in the code, interweaving tangent and primal code

Let's get started with Tapenade

Various ways to test Tapenade:

- Online via Tapenade's servlet
- Local install
- In a docker environment
- Go to XXXX TODO: Add Tiny URL once public
- Follow the README for install / usage instructions

First example

- Go through the code `01BasicTest` and write down the actual (maths) computation
- What do you expect the differentiated code should be?
- Run Tapenade from the command line *as is* and compare with your expectations.

First example

- Go through the code `01BasicTest` and write down the actual (maths) computation
- What do you expect the differentiated code should be?
- Run Tapenade from the command line *as is* and compare with your expectations.
- The log tells you it has no root $\Rightarrow$ use `-head`
- You see more variable than what you thought $\Rightarrow$ by default, every differentiable *thing* is being differentiated. Be specific in the `-head quad_fun(quad_fun/x)`:

Note that you can tell Tapenade to output the results (and log files) to an external destination

```
mkdir RESULTS
tapenade -O RESULTS
```


Take-home message 1

Useful arguments for Tapenade:

- `-O` redirects the output to a directory (needs to exist)
- `-forward` or `-tangent` or `-d` (i.e. *dot*) or `-tl` forces tapenade to work out the forward mode (which is the default)
- `-head` tells tapenade which routine you want to differentiate
- `-head ROUTINE(o1,o2)/(i1,i2,i3)` allows to compute only the necessary derivatives
- The *return variable* of the function and the function itself share the same name.

Second example

Let's look at `O2MultipleOutputs`. Its main function reads

```
double get_radius(double in, double* outcos, double* outsin){  
    double tmp = in*in/2.0+in+1.0;  
  
    *outcos = cos(tmp);  
    *outsin = sin(tmp);  
  
    return (*outcos)*(*outcos)+(*outsin)*(*outsin);  
}
```

Run `tapenade` to differentiate this differentiation *head's* output as well as the `cos` and `sin` variables with respect to the input `in`

Take-home message 2

- Differentiation tools differentiate instructions, not logic, the return value should always be 0. You may see the following piece of code:

```
return 2*(*outcos)*(*outcosd) + 2*(*outsin)*(*outsind)
```

- See how the return value is passed as an argument, while its derivative is given as the output of the diff'ed function:

```
double get_radius_d(double in, double ind, double *outcos,  
    double *outsin, double *outsind, double *get_radius) {
```

- Note the automatic naming conventions: `vard` for the (forward) differentiated variables and `functionName_d` for the (forward) differentiated functions.

How should I run the diff'ed code?

Question 1: I had a running main and now I have some functions.
What should I do?

Question 2: How can I make sure my derivatives are correct?

⇒ Use `-context` mode! It

- produces a complete differentiated program, with a sample main and helps with validation / debug harnesses.
- requires a set of utility functions within `ADFirstAidKit` available via tapenade's source code: (adapt to your path)

```
wget tapenade.inria.fr:8080/tapenade/ADFirstAidKit.tar  
tar vxvf ADFirstAidKit.tar
```

- allows debugging by running twice the compiled code with the environment variable `DBAD_PHASE=1` (perturbed run) and `DBAD_PHASE=2` (*natural* run).
- comparing both runs gives the divided difference baseline for validation

Running example 03

Run the same command as before, but provide the `-head` to the command line.

```
mkdir RESULTS  
tapenade -O RESULT/ toBeDifferentiated.c -head "get_radius(get
```

Some care should be taken:

- 1 The results are *condensed*: if the differentiation involves multiple outputs, the results need to be reduced to a single number.

- 2 Compile things via

```
gcc -I <ADFirstAidKit> toBeDifferentiated_d.c <ADFirstAidKit>
```

- 3 Typical validation sequence:

```
DBAD_PHASE=1 ./tgt  
DBAD_PHASE=2 ./tgt
```

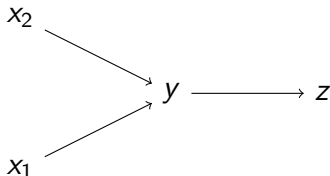
Take-home message 3

- `-context` option allows for instrumentation of the diff'ed code.
- Order of Tapenade's command line arguments doesn't matter: all non-dashed arguments are assumed to be files
- Always debug your (differentiated) code in tangent mode.
- Run the tangent context code twice, setting the `DBAD_PHASE` variable
- Compare actual tangent computed values with the provided finite differences

KEPT IF NEEDED ! Programs as Computational Graphs

Example:

```
y = x1 * x2  
z = sin(y)
```



Key idea

AD = systematic application of the chain rule on this graph.

Left to right or right to left

A program computes **out** from **in**:

$$\mathbf{in} \rightarrow v_1 \rightarrow v_2 \rightarrow \cdots \rightarrow v_9 \rightarrow \mathbf{out}$$

One can propagate $\frac{dv}{d\mathbf{in}}$ forward $\Rightarrow$ Tangent (JVP, more later):

$$1.0 = \dot{\mathbf{in}} = \frac{d\mathbf{in}}{d\mathbf{in}} \rightarrow \frac{dv_1}{d\mathbf{in}} \rightarrow \frac{dv_2}{d\mathbf{in}} \rightarrow \cdots \rightarrow \frac{dv_9}{d\mathbf{in}} \rightarrow \frac{d\mathbf{out}}{d\mathbf{in}}$$

One can propagate $\frac{d\mathbf{out}}{dv}$ backward $\Rightarrow$ Adjoint (VJP, more later):

$$\frac{d\mathbf{out}}{d\mathbf{in}} \leftarrow \frac{d\mathbf{out}}{dv_1} \leftarrow \frac{d\mathbf{out}}{dv_2} \leftarrow \cdots \leftarrow \frac{d\mathbf{out}}{dv_9} \leftarrow \frac{d\mathbf{out}}{d\mathbf{out}} = \bar{\mathbf{out}} = 1.0$$

The **initialization** are called **seeds** when dealing with many variables.

Costs of Tangent and Adjoint AD

$f : \mathbf{in} \in \mathbb{R}^p \rightarrow \mathbf{out} \in \mathbb{R}^n$, computed with cost P

$$\frac{d \mathbf{out}}{d \mathbf{in}} = \begin{pmatrix} * & * & * & * & * & * \\ * & * & * & * & * & * \\ * & * & * & * & * & * \\ * & * & * & * & * & * \end{pmatrix}$$

- $\frac{d \mathbf{out}}{d \mathbf{in}}$ costs $p * 4? * P$ using the **tangent mode**
Good if $n \geq p$
- $\frac{d \mathbf{out}}{d \mathbf{in}}$ costs $n * 4? * P$ using the **adjoint mode**
Good if $n \ll p$ (e.g $n = 1$ for a gradient)

Differentiating non high school functions

A function $F = f_p \circ f_{p-1} \circ \dots \circ f_1 : \mathbb{R}^n \rightarrow \mathbb{R}^d$ is implemented through a program

$$P: \{ I_1, I_2, \dots, I_p \}$$

Defining $V_0 = \text{in}$ and $V_k = f_k(V_{k-1})$

Reverse Mode Goal

We want gradients in high dimensional input spaces:

$$f : \mathbb{R}^p \rightarrow \mathbb{R}$$

Reverse Mode Goal

We want gradients in high dimensional input spaces:

$$f : \mathbb{R}^P \rightarrow \mathbb{R}$$

Idea

- Accumulate variations from the output
- Propagate *sensitivities* backwards

Adjoint Variables

Define for each variable v :

$$\bar{v} = \frac{\partial z}{\partial v}$$

Initialize:

$$\bar{z} = 1$$

The Aha Moment: Why $+=$?

Consider:

```
y = x*x
```

```
z = y + y
```

The Aha Moment: Why $\neq$?

Consider:

```
y = x*x  
z = y + y
```

Derivative:

$$\frac{\partial z}{\partial y} = 2$$

The Aha Moment: Why $\neq$?

Consider:

```
y = x*x  
z = y + y
```

Derivative:

$$\frac{\partial z}{\partial y} = 2$$

Backward propagation:

$$\bar{y}_+ = 1$$

$$\bar{y}_+ = 1$$

Key message

Adjoint accumulates because multiple paths contribute.

General Reverse Rule

For any variable v :

$$\bar{v} = \sum_{w \in \text{children}(v)} \bar{w} \frac{\partial w}{\partial v}$$

Interpretation

Reverse mode sums contributions from all downstream paths.

Cost of Reverse Mode

Rule of thumb

Cost $\propto$ number of outputs

Best when:

- Many inputs
- Scalar output

Common Pitfalls

- Non-differentiable functions (abs, max)
- Hidden state
- Memory in reverse mode
- Aliasing in Fortran

Takeaways

- AD applies the chain rule to programs
- Forward mode: push derivatives
- Reverse mode: pull sensitivities
- Reverse requires accumulation ($+=$)
- Tapenade automates the process

Now let's get started!

Example Fortran Code

```
double precision function energy(x)
double precision x
energy = x*x + sin(x)
end
```

Running Tapenade

Forward mode:

```
tapenade -forward -head energy energy.f
```

Reverse mode:

```
tapenade -reverse -head energy energy.f
```

What Tapenade Generates

You will see:

- Tangent variables (forward)
- Adjoint variables (reverse)
- Two-pass structure in reverse mode

Loop Example

```
do i = 1, n  
  y = y + a(i)*x(i)  
end do
```

Important

AD differentiates through loops automatically.